

Examen SLE CEAMC

Tous documents autorisés, calculatrice autorisée

Ordinateurs et autres portables/mobiles interdits

Rendre chaque partie sur des feuilles séparées

1 Application du Cell processor au traitement d'image

1.1 Communications dans le Cell processor

▷ **Question 1 :**

Quelle est la bande passante maximale théorique du bus EIB, en Gbyte/s ?

Pour répondre, donnez la formule de calcul du débit et les valeurs utilisées.

▷ **Question 2 :**

Quelle est le facteur principal limitant le débit maximal du bus à 204.8 Gbyte/s ?

▷ **Question 3 :**

Quelle est la longueur maximal d'un burst sur le bus EIB, exprimé en octets et en cycles d'horloge ?

▷ **Question 4 :**

Quels sont les 4 facteurs principaux limitant le débit sur le bus ?

Considérons un pipeline de traitement d'image constitué de 8 tâches. Chaque tâche communique avec la suivante et lui envoie un flux de données (les pixels d'une image). La première tâche lit ses données en mémoire principale et la dernière les écrits en mémoire principale.

▷ **Question 5 :**

Comment attribuer chaque tâches à un processeur SPE et chacun des flux à un canal de l'EIB afin de maximiser le débit du système ?

1.2 Architecture mémoire du Cell processor

▷ **Question 6 :**

Quelles sont les quantités de cache L1 de data et instruction du processeur principal (PowerPC) ?

▷ **Question 7 :**

Quelle est la quantité de mémoire locale d'un SPE ?

▷ **Question 8 :**

Cette mémoire locale est-elle un cache ?

▷ **Question 9 :**

Quelle est la longueur maximale d'un seul transfert DMA ?

▷ **Question 10 :**

Combien de transferts DMA peut-on enchaîner en mode liste ?

▷ **Question 11 :**

Quelles sont les deux conditions qui permettent de maximiser l'efficacité d'un transfert DMA ?

▷ **Question 12 :**

Comment un transfert DMA du MFC est-il synchronisé avec le SPU ?

▷ **Question 13 :**

Quel est le nombre minimum de cycles d'horloge nécessaires pour réaliser un transfert DMA ?

▷ **Question 14 :**

Quelles sont les unités de la figure 7 et que représente-t-elle ?

1.3 Application à la détection de visage

On souhaite réaliser un algorithme de détection de visage dans une image stockée en mémoire externe. Un pixel de l'image est codé sur 8 bits. L'image d'entrée I est de taille $t_x \times t_y$.

Les pixels sont stockés en mémoire dans l'ordre canonique. C'est-à-dire que l'adresse d'un pixel de coordonnée (x, y) de l'image I , noté $I(x, y)$, se calcule par :

$$\text{adresse}(I(x, y)) = x + y * t_x$$

Avec x variant de 0 à $t_x - 1$ (inclue) et y variant de 0 à $t_y - 1$. Le coin 0,0 est en haut à gauche et le coin $t_x - 1, t_y - 1$ est en bas à droite.

On notera qu'une zone $l \times h$ de l'image I a une largeur de l pixels et une hauteur de h pixels. La notation $(c_x, c_y) + z_x \times z_y$ désigne une zone de coin supérieur gauche de coordonnée (c_x, c_y) et de taille $z_x \times z_y$. Dans la suite, l'image a une taille de $t_x \times t_y = 512$ pixels.

L'algorithme de détection de visage commence par calculer l'image intégrale de l'image de départ. Un pixel de l'image intégrale $II(x, y)$ prend pour valeur la somme de tous les pixels du rectangle compris entre l'origine (en haut à gauche) et le pixel $I(x, y)$ de l'image de départ. Comme le dernier pixel de l'image intégrale peut prendre la valeur maximale de $255 * t_x * t_y$, il est nécessaire de coder chaque pixel de l'image intégrale sur 32 bits .

A partir de l'image intégrale, la détection de visage consiste à parcourir tous les pixels de l'image intégrale, (d_x, d_y) , pour tester si un visage est présent dans la zone $(d_x, d_y) + v_x \times v_y$. La détection retourne une valeur booléenne (détecté / non détecté) pour chaque pixel (d_x, d_y) et a besoin de tous les pixels de l'image intégrale dans la zone $(d_x, d_y) + v_x \times v_y$. Typiquement, la zone a une taille de $v_x \times v_y = 24 \times 24$.

▷ **Question 15 :**

Quelle est la taille en octets de l'image intégrale ?

▷ Question 16 :

Peut-elle tenir en entier dans une mémoire de SPE ? Combien de pixels de l'image intégrale peut-on mettre dans un SPE ?

▷ Question 17 :

Dans chacun des cas suivant, calculez le nombre de transfert DMA nécessaires pour charger la zone de l'image intégrale, ainsi que la longueur de chaque transfert :

- Une bande horizontale de largeur t_x et hauteur h_y (à calculer selon réponse précédente)
- Une bande verticale de largeur l_x et hauteur t_y (à calculer selon réponse précédente)
- Un carré de taille $i_x \times i_y$, de taille à calculer vous même

Pour répondre, faites attention à l'organisation des données en mémoire.

P.S. : Attention la suite dépend de la réponse à cette question

▷ Question 18 :

A partir des mesures relevées sur l'article, donnez le temps de transfert des données dans chacun des cas et en déduire la meilleure configuration.

▷ Question 19 :

En répartissant l'image intégrale par blocs sur les SPE et en remarquant que les zones de détection nécessitent des données qui se recouvrent, en déduire une méthode de parallélisation des calculs sur les SPE.

▷ Question 20 :

Selon le schéma proposé, combien de données sont-elle chargées plusieurs fois ?

2 Déroulement de boucles et ordonnancement d'instructions

On dispose d'un processeur MIPS permettant d'exécuter une instruction entière par cycle, auquel on adjoint un opérateur d'addition flottante pipelinée acceptant une nouvelle addition à chaque cycle mais d'une latence de 2 cycles (2 stalls entre l'instruction qui produit et celle qui consomme) et un opérateur de multiplication flottante pipelinée acceptant une nouvelle multiplication à chaque cycle mais d'une latence de 4 cycles. Par ailleurs, on rappelle que le registre résultat d'un chargement à partir de la mémoire (entier, LD, ou flottant, LD.D) n'est pas disponible dans le cycle suivant, mais dans le cycle d'après (1 stall entre l'instruction de chargement et l'instruction qui consomme le registre), et que l'instruction qui suit le branchement est exécutée *inconditionnellement*. Cette instruction doit être considérée comme un *stall* si elle est un NOP.

Soit le code :

```
for (i = n; i > 0; i--) y[i] = y[i] + a*x[i];
```

Que l'on peut traduire en assembleur MIPS par

```

L.D F0, 0(R0)      % charge a, constant sur la boucle
loop: L.D F1, 0(R1)  % charge y[i]
      L.D F2, 0(R2)  % charge x[i]
      MUL.D F2, F2, F0 % calcul de a * x[i]
      ADD.D F1, F1, F2 % somme à y[i]
      ADDUI R1, R1, 8 % passe à l'y suivant
      ADDUI R2, R2, 8 % passe à l'x suivant
      ADDUI R3, R3, -1 % décrémente le compteur
      S.D  F1        % sauve le resultat
      BNEZ R3, loop  % itère si pas au bout

```

Q1 Indiquer les stalls dans le code sans le modifier et en déduire le CPI moyen des instructions de cette boucle (le NOP étant considéré comme un stall).

Q2 On décide de réordonner les instructions sans modifier le comportement afin de minimiser les stalls. Proposez une nouvelle version du programme en mettant toujours les stalls en évidence et donnez le CPI.

Q3 On désire dérouler cette boucle 2 fois afin de gagner encore en performance, l'hypothèse étant qu'elle se fait un nombre pair de fois. En repartant *de la version initiale du programme donnée dans l'énoncé* (et non de votre version optimisée en Q2), proposez une nouvelle version du programme en mettant toujours les stalls en évidence et donnez le CPI.

Q4 Finalement, on se propose de réordonner cette version déroulée (il était temps), sachant que les autres registres entiers et flottants sont disponibles à loisir. Proposez une nouvelle version du programme en mettant toujours les stalls en évidence et donnez le CPI.

3 Réseau sur puce

Cet exercice porte sur le réseau dont la topologie est donnée à la figure 1. Les hypothèses sont les suivantes : un paquet qui va d'un routeur à l'un de ses voisins (directement connecté par un lien) met 1 cycle ; les connexions entre les nœuds (liens) sont des canaux bidirectionnels, chaque canal possédant une bande passante notée b .

1. Donnez la bande passante totale du réseau, c.-à-d. la bande passante de l'ensemble des canaux, comme une fonction de n et m ;
2. Déterminer la bande passante minimale et maximale (*bisection bandwidth* qui passe sur les canaux traversés lorsque l'on coupe le réseau en *deux parties égales*.
NB : plusieurs découpages sont possibles, il faut donc déterminer ceux qui amènent à la plus grande et plus petite découpe de bande passante ;
3. Donnez le diamètre du réseau, c.-à-d. le nombre de nœuds (*hop count*) qu'il faut traverser pour qu'un paquet voyage entre les nœuds les plus distants, incluant la source *et* la destination). Donnez un exemple de tel chemin (genre $(a, b) \rightsquigarrow (u, v)$, ou (a, b) et (u, v) sont les coordonnées des nœuds) ;
4. Donnez le nombre moyens de nœuds à traverser (distance moyenne) pour un paquet du réseau en supposant une équiprobabilité de répartition.
Par ex., on peut supposer que le nœud 00 envoie un paquet à chacun des autres nœuds du réseau.

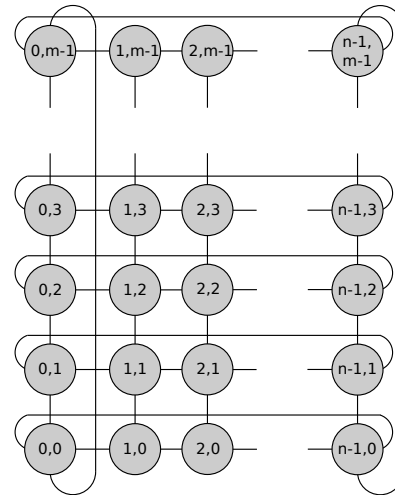


FIGURE 1 – Réseau torique non symétrique